

Auth & Session Security Analyzer

Rapport d'audit de securite Android

Application analysee : dvba
Type d'authentification : JWT
Vulnerabilites detectees : 34
Date d'analyse : 14/05/2026 23:17

Score global de securite : 0 / 10 (CRITIQUE)

1. Resume Executif

Ce resume presente les principales vulnerabilites identifiees lors de l'audit, evalue le niveau de risque global et formule les recommandations prioritaires.

L'application Android auditée est une application bancaire mobile (DamnVulnerableBank) utilisant JSON Web Tokens (JWT) pour gérer l'authentification de ses utilisateurs. L'analyse combinée statique et dynamique a mis en évidence des vulnérabilités de niveau critique qui compromettent directement la sécurité des sessions utilisateurs : les tokens JWT émis ne contiennent aucun champ d'expiration ('exp'), les rendant valides indéfiniment, et le serveur ne procède à aucune invalidation côté serveur lors de la déconnexion, permettant ainsi le rejeu de tokens après logout confirmé par une réponse HTTP 200. Par ailleurs, deux secrets cryptographiques distincts ont été retrouvés codés en dur dans le bytecode de l'application, rendant leur extraction triviale par rétro-ingénierie, et l'ensemble des tokens d'accès sont stockés en clair dans des SharedPreferences non chiffrées, exposant les credentials à toute application malveillante disposant d'un accès root ou à une extraction par sauvegarde ADB. Le niveau de risque global de l'application est évalué comme critique, avec un score particulièrement alarmant sur le domaine STORAGE (0/10) et AUTH (6/10), ce qui signifie qu'un attaquant ayant intercepté ou extrait un token peut accéder durablement aux comptes bancaires des utilisateurs sans aucune contrainte temporelle ni détection possible. Les trois recommandations prioritaires sont les suivantes : (1) corriger immédiatement la gestion du cycle de vie des tokens JWT en ajoutant un champ 'exp' court et en implémentant une liste de révocation côté serveur ; (2) migrer le stockage des tokens vers EncryptedSharedPreferences ou Android Keystore afin de protéger les credentials au repos ; (3) supprimer l'ensemble des secrets codés en dur du code source et les remplacer par des mécanismes de provisionnement sécurisé côté serveur.

2. Analyse Statique - Vulnerabilites detectees

L'analyse statique consiste en la decompilation de l'APK avec JADX et l'examen des sources Java/Kotlin par patterns regex. Les resultats sont classes par severite : CRITIQUE (-3 pts), ELEVE (-2 pts), MOYEN (-1 pt).

Fichiers analyses : 1165 | Vulnerabilites : 32 | Score : 0/10

Severite	Type	Fichier	Detail
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Parent handler not f...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager n...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager ...
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Got null intent from ...
MOYEN	Communication en clair (HTTP)	a.java	http://s
ELEVE	Identifiants dans les logs	DeviceCredentialHandler...	Log.e("DeviceCredentialHandler", "onCreate: Executor and/or call...
ELEVE	Identifiants dans les logs	DeviceCredentialHandler...	Log.e("DeviceCredentialHandler", "onActivityResult: Bridge or call...
CRITIQUE	Secret code en dur	b.java	password: "); sb.append(this.f864a.isPassword()); sb.a...
ELEVE	Token dans SharedPreferences	h.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	l.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	t.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	w.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to read GetTokenResponse ...
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to convert GetTokenRespon...

CRITIQUE	Secret code en dur	n1.java	secret="); sb.append(this.i); sb.append("
ELEVE	Identifiants dans les logs	i.java	Log.e(aVar.f1410a, aVar.a("Error parsing token claims", new Obj...
ELEVE	Token dans SharedPreferences	j.java	getSharedPreferences("com.google.firebase.auth
ELEVE	Identifiants dans les logs	k.java	Log.e(aVar2.f1410a, aVar2.a("Unable to decode token", new Obj...
ELEVE	Token dans SharedPreferences	n.java	SharedPreferences.Editor editorEdit = context.getSharedPreferences...
ELEVE	Token dans SharedPreferences	n.java	SharedPreferences.Editor editorEdit = context.getSharedPreferences...
ELEVE	Token dans SharedPreferences	p.java	getSharedPreferences(String.format("com.google.firebase.auth
ELEVE	Token dans SharedPreferences	AddBeneficiary.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ApproveBeneficiary.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	BankLogin.java	Log.d("accesstoken", string)
ELEVE	Token dans SharedPreferences	BankLogin.java	SharedPreferences sharedPreferences = BankLogin.this.getShar...
ELEVE	Token dans SharedPreferences	BankLogin.java	sharedPreferences.edit().putString("accesstoken
ELEVE	Token dans SharedPreferences	Dashboard.java	SharedPreferences.Editor editorEdit = getApplicationContext().ge...
ELEVE	Token dans SharedPreferences	Myprofile.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ResetPassword.java	getSharedPreferences("jwt
ELEVE	Token dans SharedPreferences	ResetPassword.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	SendMoney.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	ViewBalance.java	getSharedPreferences("jwt", 0).getString("accesstoken

3. Analyse Dynamique - Tests d'authentification

L'analyse dynamique evalue le comportement de l'application en cours d'exécution. Les tests portent sur la validité des tokens après déconnexion, la durée de vie, la rotation des tokens et la robustesse du mécanisme d'authentification.

Mode d'analyse :	API_DIRECT
Déconnexion effective :	Non (VULNERABLE - jeu possible)
Durée de vie du token :	None min
Rotation des refresh tokens :	Non teste

Vulnérabilités détectées lors des tests :

CRITIQUE	Le token JWT ne contient pas de champ 'exp' - il ne expire jamais
CRITIQUE	Le serveur répond HTTP 200 sur /api/user/profile après logout - invalidation côté serveur abs

4. Scores MASVS par Domaine

Les scores reflètent le niveau de sécurité par domaine OWASP MASVS v2. Chaque vulnérabilité déduit des points selon sa sévérité. Un score inférieur à 4 indique un risque critique sur ce domaine.



5. Checklist de Sécurité (MASVS)

Checklist générée par IA (Claude) adaptée au type d'authentification détecté. Chaque contrôle est mappe à un chapitre OWASP MASVS pertinent. Ces points constituent le référentiel de vérification pour la revue de code.

- [] [MASVS-AUTH-1] Vérifier que les tokens JWT contiennent un champ 'exp' valide et que la durée de vie est limitée à une valeur raisonnable (ex. 15 à 60 minutes pour un access token)
- [] [MASVS-AUTH-1] Vérifier que le serveur invalide effectivement le token côté serveur lors de la déconnexion (blacklist ou révocation de session)
- [] [MASVS-AUTH-2] Vérifier que les tokens JWT ne peuvent pas être rejoués après une déconnexion explicite de l'utilisateur
- [] [MASVS-AUTH-2] Vérifier que la rotation des tokens (refresh token) est implémentée et que les anciens tokens sont invalidés après rotation
- [] [MASVS-AUTH-3] Vérifier que l'algorithme de signature JWT utilisé est robuste (RS256 ou ES256) et que l'algorithme 'none' est explicitement rejeté par le serveur
- [] [MASVS-STORAGE-1] Vérifier que les tokens JWT (accesstoken) ne sont pas stockés en clair dans les SharedPreferences non chiffrés (fichiers c\b\alh.java, c\b\al.java, c\b\alt.java, c\b\alw.java, BankLogin.java, etc.)
- [] [MASVS-STORAGE-1] Vérifier que les tokens sont stockés dans Android Keystore ou via EncryptedSharedPreferences (Jetpack Security) plutôt que dans des SharedPreferences en mode MODE_PRIVATE non chiffré
- [] [MASVS-STORAGE-2] Vérifier qu'aucun secret cryptographique, mot de passe ou valeur sensible n'est codé en dur dans le code source (fichiers b\l\m\vb.java ligne 402 et c\c\al\fc\l1.java ligne 69)
- [] [MASVS-STORAGE-3] Vérifier qu'aucun token, identifiant ou information d'authentification n'est écrit dans les logs applicatifs (Log.d("accesstoken", string) dans BankLogin.java ligne 43 et autres occurrences)
- [] [MASVS-NETWORK-1] Vérifier que toutes les communications réseau utilisent exclusivement HTTPS/TLS et qu'aucune URL en HTTP clair n'est présente dans le code (URL http://s détectée dans al\al\al.java ligne 487)
- [] [MASVS-NETWORK-1] Vérifier que la configuration réseau Android (network_security_config.xml) interdit explicitement le trafic en clair (cleartextTrafficPermitted="false")
- [] [MASVS-CRYPTO-1] Vérifier que le secret utilisé pour la signature JWT (détecté en dur dans n1.java) est généré aléatoirement, d'une longueur suffisante (? 256 bits), et stocké de manière sécurisée côté serveur
- [] [MASVS-AUTH-1] Vérifier que les claims JWT (sub, iss, aud) sont correctement validés côté serveur à chaque requête et que les erreurs

de parsing de claims sont gérées sans exposer d'informations sensibles dans les logs

- [] [MASVS-STORAGE-2] Vérifier que les tokens Firebase (com.google.firebase.auth) stockés dans les SharedPreferences sont également protégés par chiffrement et soumis aux mêmes règles de cycle de vie que les tokens JWT applicatifs

6. Criteres d'Acceptation de Securite

Les criteres d'acceptation definissent les conditions que le systeme doit satisfaire pour etre considere comme securise. Ils servent de base aux tests d'acceptation, aux revues de code et aux audits de conformite MASVS.

1. Le système DOIT émettre des tokens JWT contenant un champ 'exp' avec une durée de validité maximale de 60 minutes afin d'éviter l'utilisation indéfinie d'un token compromis.
2. Le système NE DOIT PAS retourner une réponse HTTP 200 sur un endpoint authentifié après que l'utilisateur s'est déconnecté, afin de garantir l'invalidation effective des sessions côté serveur.
3. Le système DOIT stocker les tokens JWT exclusivement dans EncryptedSharedPreferences ou via Android Keystore afin de protéger les tokens contre toute extraction depuis un appareil rooté ou une sauvegarde non chiffrée.
4. Le système NE DOIT PAS inscrire de tokens, de mots de passe ou de toute valeur d'authentification dans les journaux applicatifs (Logcat) afin d'empêcher la fuite de credentials via les logs accessibles à d'autres applications sur un appareil déboggué.
5. Le système NE DOIT PAS contenir de secrets cryptographiques ou de mots de passe codés en dur dans le bytecode de l'application afin d'éviter leur extraction par rétro-ingénierie statique.
6. Le système DOIT implémenter une rotation des refresh tokens à usage unique (one-time use) afin de détecter et bloquer toute tentative de rejeu d'un token volé.
7. Le système DOIT utiliser exclusivement des connexions HTTPS avec TLS 1.2 minimum pour toutes les communications réseau afin d'empêcher l'interception des tokens JWT en transit.
8. Le système DOIT valider l'ensemble des claims JWT (exp, iss, aud, sub) côté serveur à chaque requête authentifiée afin de rejeter tout token malformé, expiré ou émis par une source non autorisée.
9. Le système NE DOIT PAS utiliser l'algorithme de signature JWT 'none' et doit rejeter explicitement toute tentative de validation d'un token signé avec cet algorithme, afin de prévenir les attaques de contournement de signature.